

Engineering Intern Assessment

Project Sentinel

Full-Stack Engineering · Golang / React / DevOps

POSITION	Full-Stack Engineering Intern	EXPECTED DURATION	48 – 72 Hours
----------	-------------------------------	-------------------	---------------

Overview

Our systems must be **resilient, scalable, and highly observable**. For this assessment you will step away from standard CRUD applications and build **Project Sentinel**: a high-performance API Multiplexer with a custom Circuit Breaker, complete with a real-time visualisation dashboard and chaos-tested infrastructure. We are looking for engineers who care about *memory management, concurrency, and developer experience*.

The Scenario

Your system will sit between incoming client requests and downstream APIs. It must route a continuous stream of traffic to a **Primary API**.

If the Primary API becomes unresponsive, times out, or fails, your system must instantly **trip a circuit breaker** and reroute traffic to a **Secondary (Fallback) API** — without dropping the client's request.

You must build the core routing logic, a frontend to monitor the chaos, and the infrastructure to run and test it all.

Technical Requirements

1 · Backend (Golang)

- **The Router:** Build an HTTP reverse proxy/router using Go's standard library (net/http).
- **Custom Circuit Breaker:** Implement the Circuit Breaker pattern from scratch — do NOT use external resilience libraries. It must transition between Closed, Open, and Half-Open states based on configurable error thresholds.
- **Context Timeouts:** If the Primary API takes longer than exactly 200 ms to respond, the request context must cancel and the system must seamlessly fall back to the Secondary API.
- **Memory Constraint:** The Go service must be highly optimised. It will run in a Docker container with a hard memory limit of 128 MB. You must manage Goroutines and channels efficiently to avoid OOM crashes under load.
- **Telemetry:** Stream real-time metrics (RPS, active routes, circuit breaker state) over WebSockets.

2 · Frontend (React / JavaScript)

- **The "War Room" Dashboard:** Build a React application that connects to the backend via WebSockets to display system health.

- **High-Frequency Rendering:** The backend will send dozens of updates per second. Use techniques like virtualisation or optimised state management to ensure the browser does not freeze or lag.
- **Visual State Machine:** The UI must visually indicate the state of the Go backend. If the circuit trips to "Open", the UI should reflect the failure and visually show traffic routing to the Fallback API.

3 · DevOps & Chaos Engineering

- **Containerisation:** Provide a docker-compose.yml that spins up the entire stack — your Go Backend, React Frontend, and two dummy API containers (Primary and Secondary).
- **Multi-Stage Builds:** Your Go Dockerfile must use multi-stage builds to keep the final image size minimal.
- **Chaos Testing (Toxiproxy):** Integrate Toxiproxy into your docker-compose network. Provide a script or instruction to inject 500 ms of latency and a 20 % packet drop rate into the Primary API to prove your circuit breaker works under hostile conditions.

Evaluation & Scoring Criteria

We evaluate beyond just "does it work" — we look at **how** it works.

#	Area	What We Look For
1	Concurrency & Efficiency (Backend)	Are Goroutines handled safely? Memory leaks? Does it survive the 128 MB limit under a load test?
2	Frontend Performance	Is the React code well-structured? Are unnecessary re-renders avoided during the WebSocket data stream?
3	Resilience (DevOps)	Does Docker Compose spin up flawlessly? Does the system successfully reroute traffic when Toxiproxy injects latency?
4	Code Quality	Clean architecture, meaningful variable names, proper error handling, and logical commit history.

Deliverables & Submission

- **Source Code:** A link to a public GitHub repository.
- **Documentation:** A comprehensive README.md containing — instructions to start the environment via Docker, instructions to trigger the Toxiproxy chaos test, and a brief explanation of your architectural choices (Circuit Breaker & React rendering).
- **Video Walkthrough (Optional but Recommended):** A 3-minute Loom video demonstrating the UI, triggering the Toxiproxy failure, and showing the system recovering.

This assessment is designed to reflect real engineering challenges. Focus on correctness, clarity, and resilience over feature breadth.